

Speech Command Recognition: A Deep Learning Approach

Alberto Calabrese

Department of Mathematics, University of Padova

alberto.calabrese.2@studenti.unipd.it

Abstract—Small-Footprint Keyword Spotting (KWS) is a critical technology for enabling voice-based interaction on resource-constrained devices. Selecting the optimal deep learning architecture suitable for this task involves navigating complex trade-offs between performance and efficiency. This paper presents a systematic, comparative study of diverse deep learning paradigms for KWS, evaluating a spectrum of models from established CNNs to modern Hybrid and Transformer architectures on the Google Speech Commands dataset. Our primary contributions are a holistic analysis of these architectural trade-offs and the development of a novel experimental model that synthesizes a ResNet backbone, Squeeze-and-Excitation attention, and a GAN-inspired classifier. Our results show that this novel architecture achieves the highest classification performance with a mean macro F1-score of 0.947. While our analysis reveals that other models like the Hybrid CNN-BiLSTM offer a superior balance of performance and efficiency, we selected our top-performing experimental model for deployment to validate the upper bound of our work. Its successful conversion to TensorFlow Lite and integration into a real-time Android application confirms its practical viability, providing a blueprint for developing highly accurate on-device KWS systems.

Index Terms—Keyword Spotting, Deep Learning, Convolutional Neural Networks, Transformer Architecture, Attention Mechanisms, On-Device AI.

I. INTRODUCTION

Voice-based interaction has become a cornerstone of modern human-computer interfaces, enabling intuitive, hands-free control over a vast ecosystem of devices. At the core of this paradigm is the keyword spotting (KWS) task, which involves the detection of specific spoken commands within an audio stream. The demand for efficient and accurate KWS systems, particularly those with a “small footprint” suitable for on-device processing on IoT and mobile hardware, has driven significant research and development. Early approaches to KWS relied on traditional signal processing and machine learning techniques. However, the advent of deep learning has caused a paradigm shift in the field. Convolutional Neural Networks (CNNs) established a strong performance baseline by treating audio spectrograms as images [1]. While effective, this approach often overlooks the explicit temporal nature of speech. Consequently, more complex architectures incorporating Recurrent Neural Networks (RNNs) or state-of-the-art Transformer models [2] have been explored. However, a significant challenge persists: Selecting the optimal architecture involves navigating a complex landscape of trade-offs between accuracy, computational complexity, model size, and inductive bias. There is no universally superior paradigm, and

the best choice is contingent on the specific constraints of the application. This paper presents a systematic, comparative investigation into the application of diverse deep learning paradigms for small-footprint KWS. Our work moves beyond a singular focus on performance, providing a holistic analysis of architectural trade-offs. We implement and evaluate a wide spectrum of models on the Google Speech Commands dataset [3], from baseline CNNs and deep residual networks to hybrid recurrent and attention-based models, including a pure Vision Transformer. The primary contribution of this work is a comprehensive comparison of these paradigms, the development of a novel high-performance architecture that synthesizes concepts from multiple domains, and the validation of our findings through practical implementation. Critically, we demonstrate this applicability by deploying our **highest-performing experimental model** in a real-time Android application to showcase the upper bound of our research. The main contributions of this work are summarized as follows:

- We conduct a comprehensive comparative study of five distinct deep learning architectures for keyword spotting: a Baseline CNN, a Deep Residual Network (ResNet), a Hybrid CNN-BiLSTM, a Hybrid CNN-Transformer, and a pure Vision Transformer (ViT).
- We propose a novel experimental architecture that augments a ResNet backbone with a Squeeze-and-Excitation attention mechanism and a classifier head inspired by GAN discriminators, which achieves the **highest classification performance** among all models tested.
- We perform an ablation study on our Transformer-based models by substituting standard Layer Normalization with a recent normalization-free technique, providing insights into the effectiveness of cutting-edge methods on this task.
- We demonstrate the real-world viability of our research by successfully deploying our **top-performing model** in a functional Android application for on-device inference, validating its applicability despite its complexity.

This report is structured as follows. In Section II we review the state-of-the-art. The complete methodology, including data preprocessing and detailed architectural descriptions, is presented in Section III. Section IV provides a detailed evaluation and comparison of the models. Finally, Section V concludes the paper and discusses potential future work.

II. RELATED WORK

The field of small-footprint Keyword Spotting (KWS) has evolved significantly with the advent of deep learning. This section reviews the key architectural paradigms and techniques that form the foundation of our work, highlighting both their contributions and their limitations to motivate our comparative study.

A. Convolutional Architectures for KWS

The application of Convolutional Neural Networks (CNNs) represented a pivotal moment for KWS, shifting the paradigm from traditional methods to end-to-end deep learning. A foundational study by Sainath and Parada [1] demonstrated that even relatively shallow CNNs, when applied directly to log-Mel spectrograms, could achieve state-of-the-art results. This established the spectrogram-as-image methodology, which remains a strong baseline. To overcome the limitations of shallow networks, deeper architectures based on residual learning [4] were introduced. Tang and Lin [5] specifically adapted this for KWS, showing that Deep Residual Networks (ResNets) could significantly improve accuracy by enabling the training of much deeper models without suffering from gradients vanishing. While these CNN-based models provide a powerful inductive bias for local spectro-temporal patterns, they lack an explicit mechanism for modeling long-range dependencies. Our work addresses this limitation by systematically comparing these strong convolutional models against hybrid and pure attention-based architectures.

B. Attention Mechanisms and Transformer Models

The introduction of the attention mechanism, first popularized in machine translation by Vaswani et al. [2], has fundamentally reshaped sequence modeling. The Transformer architecture, based entirely on self-attention, dispensed with recurrence and convolution, enables parallel computation and the ability to capture global dependencies across sequences. Early works adapted attention for speech recognition [6] and specifically for KWS [7], often in hybrid systems that incorporated Recurrent Neural Networks (RNNs). The "Speech-Transformer" [8] further demonstrated the viability of end-to-end, non-recurrent models for speech tasks. More recently, the Vision Transformer (ViT) [9] has further expanded this paradigm to the domain of computer vision by treating images as sequences of patches. These works demonstrate the immense potential of attention, but also highlight its drawbacks, such as a reduced inductive bias and a high demand for data, which can be challenging in a KWS context without extensive pre-training. Our study provides a direct, empirical comparison of these Transformer-based approaches against strong CNN and recurrent models on the same benchmark.

C. Advanced Techniques and Our Contribution

Beyond the fundamental architectures, our work incorporates several advanced techniques. We employ SpecAugment [10], a simple yet highly effective data augmentation method that operates on the spectrogram by masking blocks of

frequency and time, improving model generalization. Our experimental model also integrates Squeeze-and-Excitation (SE) networks [11], a form of channel-wise attention that enables a model to adaptively re-weight its feature channels, and draws inspiration from the discriminator design of Generative Adversarial Networks (GANs) [12] for its robust classifier. Furthermore, we explore cutting-edge normalization-free techniques for Transformers [13] to investigate their impact on performance and efficiency.

III. METHODOLOGY

This section presents the complete technical methodology of our study. We first describe the data preprocessing pipeline engineered to transform audio inputs into feature representations. Subsequently, we present the architectural design of the different models under investigation, from the initial baseline to our final experimental network. The section concludes by outlining the process for deploying a trained model into a real-time Android application for real-time inference.

A. Data Preprocessing

The raw audio data from the Google Speech Commands V2 dataset [3] requires significant preprocessing to be suitable for training deep learning models. The pipeline has been engineered to transform raw audio clips into a fixed-size feature representation while augmenting the training data to enhance model generalization.

1) *Dataset Partitioning:* To ensure a fair and reproducible evaluation, the dataset was split into training, validation, and test sets following the official procedure recommended by the dataset owners. This method uses a deterministic hash of the filename to assign each audio file to a partition. We assigned 10% of the files for validation, 10% for testing, and the remaining 80% for training. This approach is designed to prevent contamination between sets and ensure that all utterances from a single speaker are not split across different partitions.

2) *Feature Extraction:* The core of our preprocessing pipeline is the conversion of raw audio waveforms into a more informative time-frequency representation. For each .wav file, sampled at 16,000 Hz, we compute a log-Mel spectrogram (Fig.1). This process involves:

- **Framing:** The audio signal is segmented into short frames of 25 ms, with a stride of 10 ms between consecutive frames.
- **Mel-Spectrogram Computation:** For each frame, a Short-Time Fourier Transform (STFT) is computed. The resulting power spectrum is then mapped onto the Mel scale using a filter bank of 40 triangular filters, producing a Mel spectrogram. This representation is perceptually more relevant as it mimics the non-linear frequency response of the human ear.
- **Logarithmic Scaling:** The Mel spectrogram's amplitude is converted to a logarithmic scale, which helps to balance the dynamic range and better captures the characteristics of human speech.

- **Fixed-Size Input:** To ensure a consistent input shape for the neural networks, the spectrograms are truncated or padded along the time axis to obtain a constant dimension of 40×101 (*Mel bins* $\times$ *time frames*).

The final processed feature representation is shown in Fig. 1.

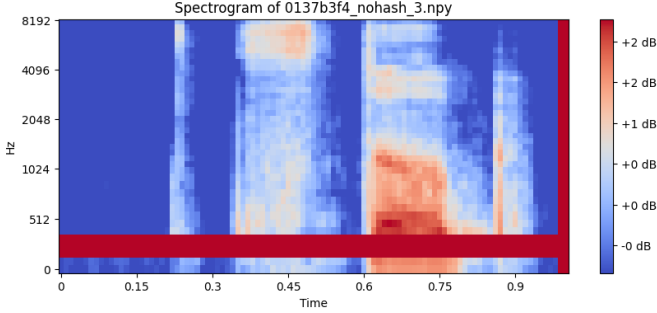


Fig. 1: Example of a log-Mel spectrogram after preprocessing. The solid horizontal red band is a visual artifact resulting from a frequency mask applied by the SpecAugment data augmentation technique, followed by Z-score normalization.

3) *Data Augmentation:* To prevent overfitting and enhance the robustness of our models, we apply several data augmentation techniques exclusively to the training set. These augmentations are applied stochastically during the preprocessing phase.

- **Noise Addition:** With a probability of 0.3, we mix a random segment from the provided background noise clips into the audio signal. To simulate varying intensities of noise, the amplitude of the noise segment is scaled by a random factor uniformly chosen from the range $[0.1, 0.4]$ before being added.
- **Time Shifting:** Audio signals are randomly shifted forward or backward in time by up to 100 ms with a probability of application of 0.3. This encourages the model to become invariant to the exact position of the keyword within the time window.
- **SpecAugment:** We apply a simplified version of SpecAugment [10] directly on the Mel spectrogram, still with a 0.3 probability. This involves masking out random contiguous blocks of frequency channels (frequency masking) and time steps (time masking). This process, after normalization, is often visible as solid colored bands in the resulting spectrograms, as seen in Fig. 1.

4) *Normalization and Dataset Creation:* After augmentation, each spectrogram is normalized using Z-score normalization on a per-sample basis. This step is crucial for stabilizing the training process. To handle the class imbalance inherent in the dataset, we calculated class weights using the “balanced” mode from scikit-learn, which assigns higher weights to under-represented classes. These weights are later used in the loss function during model training. The datasets are batched with a size of 64 and the prefetch operation is used to optimize the input pipeline for GPU training.

B. Baseline CNN

To establish a strong performance benchmark, our initial model is a modern Convolutional Neural Network (CNN). The design is conceptually inspired by the pioneering work of Sainath and Parada [1], but updated with modern best practices for efficiency. The goal is to implement a straightforward, yet effective architecture to validate the data pipeline and set a high-quality baseline for our subsequent, more complex models.

1) *Feature Extractor:* The extractor processes the input log-Mel spectrogram of size $40 \times 101 \times 1$. It consists of two sequential Conv2D layers with 32 and 64 filters, both using a 3×3 kernel and the Rectified Linear Unit (ReLU) activation function. Each convolutional layer is followed by BatchNormalization to stabilize gradients and accelerate training. A single MaxPooling2D layer with a 2×2 pool size is applied after the second convolutional block to reduce the spatial dimensionality of the feature maps and introduce a degree of translation invariance.

2) *Classifier Head:* A key feature of this baseline is its efficient classifier head. Instead of a naive Flatten layer, which would create more than 7 million parameters, a GlobalAveragePooling2D layer is employed. This operation averages each of the 64 feature maps from the extractor into a single value, drastically reducing the number of parameters and acting as a strong structural regularizer against overfitting. The resulting 64-dimensional vector is then passed through a hidden Dense layer with 128 neurons, which is regularized by an aggressive Dropout rate of 0.5, before the final softmax output layer.

3) *Architectural Considerations:* The adoption of GlobalAveragePooling2D transforms the baseline into a lightweight and highly effective model, with only **32k trainable parameters**, capable of achieving a respectable test accuracy of approximately 75.5%. This strong result sets a challenging benchmark for our more advanced architectures. However, the model’s primary limitations are its shallow nature, which restricts its ability to learn deeply hierarchical features, like those in a ResNet, and its lack of an explicit mechanism for modeling long-range temporal dependencies. Recurrent or attention-based backends are specifically designed for this task.

C. Residual CNN

While our Baseline CNN proved effective, its shallow nature inherently limits its capacity to learn more complex and abstract features. To overcome this limitation, we implemented a Residual CNN (ResNet), an architecture specifically designed to enable the training of much deeper networks. Based on the principles of Deep Residual Learning [4] and its successful application to KWS [5], our objective is to determine whether a deeper feature hierarchy can produce a significant performance improvement over the strong baseline.

The model, depicted in Fig. 2, consists of a standard convolutional stem, a stack of custom residual blocks, and an efficient classifier head.

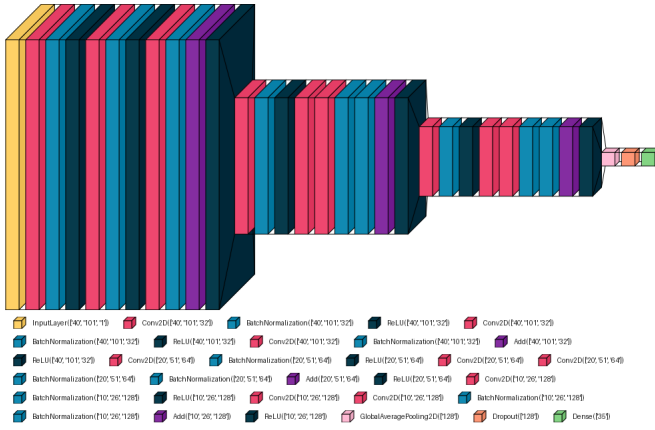


Fig. 2: Block diagram of the Residual CNN architecture.

1) *The Residual Block*: The core component of the model is the residual block. It is designed to learn a residual function with reference to the block’s input, rather than learning the underlying mapping directly. The main path of each block contains two consecutive 3×3 Conv2D layers. A shortcut connection bypasses these layers, and its output is added element-wise to the output of the main path. This structure is critical for enabling stable gradient flow in deeper networks, mitigating the vanishing gradient problem and allowing for effective training of the increased architectural depth. To handle changes in dimensionality, the shortcut connection is projected using a 1×1 Conv2D layer.

2) *Network Structure:* The overall network begins with a 3×3 convolutional stem layer with 32 filters. This is followed by a stack of three residual blocks that progressively downsample the feature maps (using a stride of 2) while doubling the filter depth (32, 64, then 128). This design creates a hierarchical feature pyramid, allowing the model to learn patterns at multiple scales: early blocks capture fine-grained local features, while later blocks learn more abstract, large-scale structures within the spectrogram.

3) *Classifier Head*: Similarly to our baseline, the classifier head employs a GlobalAveragePooling2D layer to efficiently aggregate the features from the final convolutional block. This ensures that the model remains parameter-efficient despite its depth. The resulting vector is then passed through a Dropout layer for regularization before the final Dense layer with a softmax activation performs the classification. This architecture, while significantly deeper than the baseline, maintains a lean profile with approximately 312k trainable parameters.

D. Hybrid CNN + BiLSTM

To explicitly model the temporal nature of speech, we developed a hybrid architecture that synergistically combines a Convolutional Neural Network (CNN) with a Bidirectional Long Short-Term Memory (BiLSTM) network. This approach is motivated by a well-established paradigm in speech processing, where a convolutional frontend extracts robust, local spectro-temporal features, and a recurrent backend models the contextual relationships over time [14]. The proposed model

follows a three-stage pipeline: a CNN feature extractor, a specialized interface to reshape the features for the recurrent layer, and a BiLSTM-based classifier.

1) *CNN Feature Extractor*: The initial stage processes the $40 \times 101 \times 1$ input spectrogram. It consists of two convolutional blocks with 32 and 64 filters, respectively, both employing 3×3 kernels and ReLU activation. Each convolutional layer is followed by BatchNormalization for training stability and MaxPooling2D for downsampling. This powerful frontend reduces the input dimensionality while learning a rich hierarchy of local acoustic patterns, resulting in a compact feature map of size $10 \times 25 \times 64$.

2) *CNN-to-RNN Interface*: A critical step in this hybrid architecture is preparing the 2D feature map for the sequential processing of the RNN.

- **Sequencing:** The $10 \times 25 \times 64$ feature map is first reordered using a Permute layer to establish the temporal axis as the sequence dimension. A Reshape layer then flattens the frequency and channel dimensions, transforming the map into a sequence of 25 time steps, each with a 640-dimensional feature vector (10×64).
- **Feature Compression:** To create a more compact representation for the recurrent layer, a TimeDistributed (Dense) layer is applied. This layer projects each of the 25 feature vectors from a dimension of 640 to 128, significantly reducing the complexity of the model and the computational load for the subsequent BiLSTM layer.

- **Feature Compression:** To create a more compact representation for the recurrent layer, a TimeDistributed (Dense) layer is applied. This layer projects each of the 25 feature vectors from a dimension of 640 to 128, significantly reducing the complexity of the model and the computational load for the subsequent BiLSTM layer.

3) *BiLSTM Modeler and Classifier*: The core of the temporal modeling is a Bidirectional wrapper around an LSTM layer with 32 units. The bidirectional nature allows the network to process the sequence in both forward and backward directions, enabling each time step’s output to be informed by both past and future context. The BiLSTM layer processes the input sequence and outputs a corresponding sequence of hidden states. This entire sequence is then passed to a GlobalAveragePooling1D layer, which aggregates it into a single, fixed-size feature vector. Finally, a classifier head, composed of a dense hidden layer and regularized with Dropout, processes this vector to produce the final softmax probability distribution.

E. Hybrid CNN + Transformer

Continuing on the hybrid model concept, we designed a more modern architecture to directly compare a recurrent backend against a Transformer-based one. This model aims to leverage the self-attention mechanism, as introduced by Vaswani et al. [2], to capture global dependencies in a highly parallelizable manner, representing the state-of-the-art in many sequence modeling tasks [8].

1) *CNN Frontend and Sequencing*: The model employs a CNN frontend that is identical to the one described for the BiLSTM model in the previous section III-D. It processes the input spectrogram to produce a sequence of 25 feature vectors, each with a dimension of 640. This allows a direct comparison of the subsequent backend modules.

2) *Transformer Embedding and Encoder*: Before being processed by the Transformer, the sequence of feature vectors requires two additional steps. First, a Dense layer projects the 640-dimensional vectors into the Transformer’s smaller embedding space (dimension 64). Second, a custom PositionalEncoding layer injects sinusoidal positional information, which is essential for the permutation-invariant self-attention mechanism to understand sequence order.

The resulting sequence of enriched embeddings is then processed by a stack of two standard Transformer encoder blocks. Each block contains two main sub-layers: a Multi-Head Self-Attention layer and a position-wise Feed-Forward Network. Both sub-layers employ residual connections and LayerNormalization, which are essential for stable training.

In an experimental ablation, we also explored replacing the LayerNormalization with a Dynamic Tanh nonlinearity, as proposed by Zhu et al. in their work on normalization-free Transformers [13]. While this variant resulted in a marginal reduction in model size and inference time, it led to a notable degradation in classification performance on the test set (an accuracy of 89.2% compared to 93.6%). Consequently, we retained the standard LayerNormalization in our final architecture.

3) *Classifier*: The sequence of output vectors from the final Transformer block is aggregated into a single representation using GlobalAveragePooling1D. This vector is then passed through a small Multi-Layer Perceptron, regularized with Dropout, which terminates in a softmax layer to produce the final class probabilities.

F. Vision Transformer (ViT)

To conclude our exploration of established architectural paradigms, we implemented a Vision Transformer (ViT), representing a significant shift from the previous models. This approach, directly inspired by the application of Transformers to computer vision by Dosovitskiy et al. [9], completely discards the convolutional inductive biases present in all our other models. The core philosophy is to treat the 2D log-Mel spectrogram as a sequence of smaller, non-overlapping image patches, aiming to determine if a pure, self-attention-based architecture can effectively learn the required spectro-temporal patterns without a convolutional frontend.

1) *Spectrogram to Embedded Patch Sequence*: The transformation from a 2D spectrogram into a sequence of enriched embeddings suitable for the Transformer involves three key steps:

- **Patch Extraction**: The standard ViT architecture requires input dimensions to be divisible by the patch size. We first resize the 40×101 input spectrogram to 40×104 . A custom Patches layer then partitions this image into a grid of $5 \times 13 = 65$ non-overlapping patches of size 8×8 .
- **Linear Projection**: Each of these 2D patches is then flattened into a vector and linearly projected into the Transformer’s latent embedding space by a trainable Dense layer.

- **Positional Embedding**: To provide the permutation-invariant self-attention mechanism with crucial spatial awareness, we employ learnable positional embeddings. A unique vector is learned for each of the 65 patch positions and is added to the corresponding patch embedding.

2) *Transformer Encoder*: The resulting sequence of 65 enriched patch embeddings is then processed by a stack of two standard Transformer encoder blocks, identical in structure to those used in our hybrid model (Sec. III-E). Each block uses Multi-Head Self-Attention to weigh the importance of all other patches when encoding a specific patch, allowing the model to capture global relationships across the entire spectrogram from the very first layer. As with the hybrid model, we also experimented with a normalization-free variant using Dynamic Tanh [13]. This again resulted in lower performance (an accuracy of 81.6% vs. 85.2%), reinforcing the effectiveness of standard LayerNormalization for these architectures on our dataset.

3) *Classifier Head*: To produce a final classification, the sequence of output embeddings from the final Transformer block is aggregated into a single feature vector using GlobalAveragePooling1D. This vector is then passed through a small Multi-Layer Perceptron (MLP) head, consistent with our other Transformer-based model. This head consists of a hidden Dense layer with ReLU activation, followed by Dropout for regularization, before terminating in a final softmax layer for class probability prediction.

G. Experimental: Residual Attention Network with Discriminator-Style Classifier

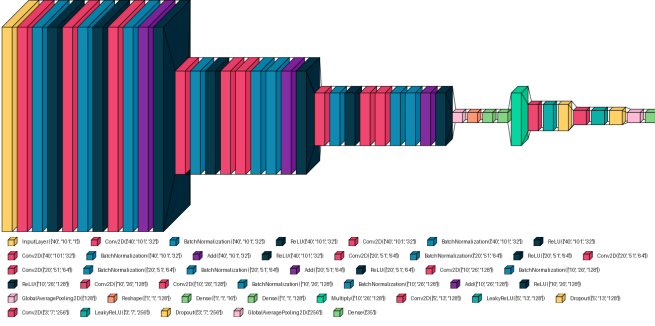
To push the performance boundary for keyword spotting, we developed a novel, experimental architecture that synthesizes concepts from different deep learning domains. The design philosophy is to augment a powerful ResNet backbone with advanced mechanisms for feature refinement and classification. The architecture, illustrated in Fig. 3, integrates three key ideas:

- 1) A **ResNet backbone** for robust, hierarchical feature extraction.
- 2) A **Squeeze-and-Excitation (SE) block** for dynamic, channel-wise feature recalibration, as proposed by Hu et al. [11].
- 3) A **classifier inspired by modern GAN discriminators**, specifically designed to be powerful classifiers [12].

The objective is to create a model that not only learns powerful features but also learns to focus on the most informative among them for a given input.

1) *ResNet Feature Extractor*: The model’s feature extractor is identical to the ResidualCNN architecture described in Sec. III-C. It leverages a convolutional stem and a stack of three residual blocks to process the input spectrogram, providing a strong foundation of rich multi-scale features and returning a high-level feature map of size $10 \times 26 \times 128$.

2) *Squeeze-and-Excitation Attention*: After the ResNet backbone, a Squeeze-and-Excitation (SE) block is inserted to perform dynamic, channel-wise feature recalibration. This



- **Efficiency Metrics:** Number of trainable parameters, total model size on disk (in Megabytes), and average inference time per sample on a Tesla P100 GPU, reflecting the model’s throughput in a batched environment.

We also designed two complementary aggregate scores based on the mean of the metrics to facilitate a deeper analysis of the trade-off. The **Normalized Score** preserves the magnitude of differences between models. For each metric m and each model, let m_s be the value obtained for seed $s \in \{42, 1999, 2025\}$. We first compute the mean value $\mu_m = \frac{1}{3} \sum_{s=1}^3 m_s$, this value is then normalized to a $[0, 1]$ range using Min-Max scaling, denoted as $\hat{\mu}_m$. The score is then calculated as a weighted sum, as shown in Equation 1:

$$Norm. Score = \sum_{m \in \mathcal{P}} w_m \hat{\mu}_m + \sum_{c \in \mathcal{C}} w_c (1 - \hat{\mu}_c) \quad (1)$$

where $\mathcal{P}$ is the set of performance metrics and $\mathcal{C}$ is the set of cost (efficiency) metrics, with corresponding weights w . This formula rewards high performance while penalizing high resource costs. The **Rank Score**, is a non-parametric score robust to outliers. It is calculated by first ranking models for each metric, then averaging the ranks to produce a *Performance Score* and an *Efficiency Score*, which are finally combined with a 60/40 weighting. For this score, a lower value is better.

B. Overall Performance and Trade-off Analysis

The comparative results for all five advanced models are summarized in Table 1, sorted by our primary performance metric, the F1-score. The multi-seed evaluation reveals a compelling narrative about the architectural efficacy, with clear trade-offs between accuracy and efficiency, further visualized in Fig. 4 and Fig. 5.

The Baseline CNN, with a F1-score of 0.74, establishes a strong and meaningful benchmark. The results (Table 1) clearly show that all advanced architectures significantly surpass this baseline. The top tier of performance is dominated by models with deep convolutional backbones. Our experimental model emerges as the undisputed leader in raw classification performance, achieving the highest test accuracy (95.30%) and F1-score (0.947). This model has an extremely low standard deviation ($\sigma = 0.003$) which highlights its remarkable training stability. This validates our hypothesis that augmenting a strong ResNet foundation with channel-wise attention and a specialized classifier yields measurable performance gains. The standard Residual CNN is a very close second, confirming that deep hierarchical feature extraction is a powerful strategy for this task. Hybrid CNN-BiLSTM also proves to be highly competitive, positioning itself firmly in the top-tier performance category.

However, these top-performing models come at a cost. As shown in the figures (4 & 5), they are the largest and have the highest inference latency. At the other end of the spectrum, the Hybrid CNN-Transformer and Vision Transformer consistently exhibit superior inference speed, highlighting the high degree of parallelization inherent of the

Transformer-based models. The aggregate scores distill this trade-off between performance and efficiency. The Hybrid CNN-BiLSTM achieves the highest *Normalized Score* (0.89) and the best (lowest) *Rank Score* (2.87), showing a remarkable compromise between top-tier performance and outstanding efficiency, being the lightest model in terms of parameters (after the shallow Baseline CNN).

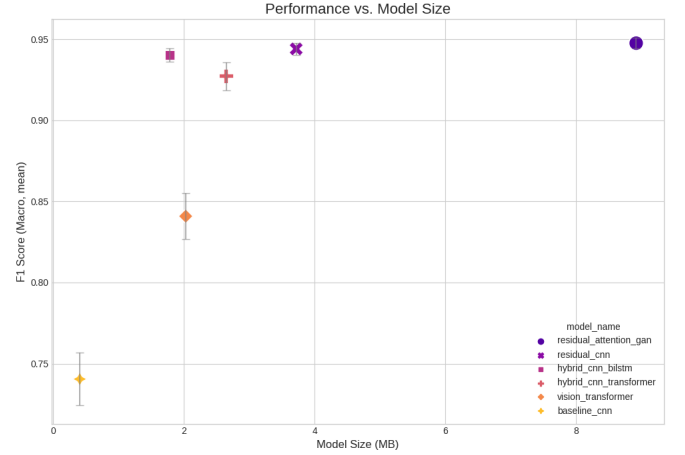


Fig. 4: Trade-off between F1-score and model size.

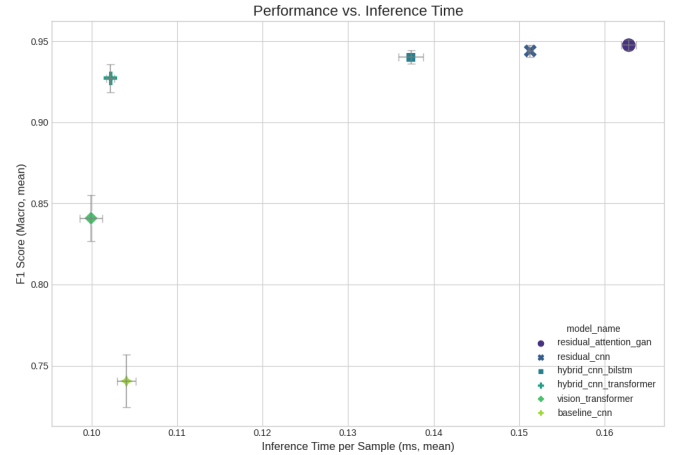


Fig. 5: Trade-off between F1-score and inference time.

C. In-depth Analysis and Deployment of the Best Performing Model

To demonstrate the maximum potential of our architectural explorations, we selected the model with the highest F1-score, our **Experimental Model**, for a deeper qualitative analysis and for deployment. While other models like the Hybrid CNN-BiLSTM offer a better balance, this choice allows us to validate that even our most complex experimental model can be successfully implemented in a real-world scenario.

1) *Confusion Matrix*: The confusion matrix for our experimental model, shown in Fig. 6, provides a granular view of its classification performance. The strong concentration of

TABLE 1: Overall performance and efficiency comparison of all competitive models on the test set, sorted by F1-score. The best value in each column is highlighted in bold.

Model	Acc. (%)	F1	Tr. Params	Size (MB)	Inf. (ms)	Norm. Score $\uparrow$	Rank Score $\downarrow$
Experimental Model	95.30	0.947	764,643	8.92	0.163	0.800	3.00
Residual CNN	95.00	0.944	313,315	3.72	0.151	0.866	3.20
Hybrid CNN-BiLSTM	94.57	0.940	148,899	1.79	0.137	0.894	2.87
Hybrid CNN-Transformer	93.56	0.927	216,547	2.65	0.102	0.883	3.73
Vision Transformer	85.22	0.841	164,643	2.03	0.099	0.572	3.93
Baseline CNN	75.55	0.740	32,035	0.41	0.104	0.193	4.26

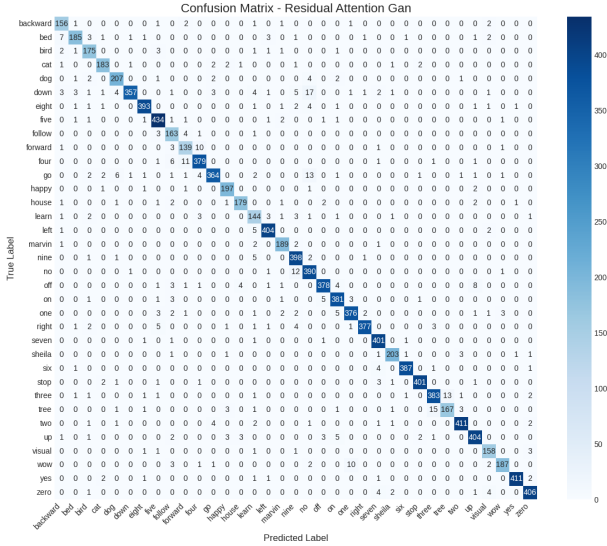


Fig. 6: Confusion matrix for the Experimental model.

values along the main diagonal confirms its high accuracy. The few notable off-diagonal entries reveal predictable phonetic confusions. For instance, the model occasionally confuses words with similar vowel structures like "down" and "no", or similar pairs such as "tree" and "three". The overall clarity of the matrix, however, underscores the model's robust classification capability.

2) *T-SNE Visualization*: The t-SNE plot in Fig. 7 visualizes the high-dimensional feature space learned by the model's penultimate layer. The clear separation of most classes into distinct and relatively tight clusters demonstrates that the network is successfully learning discriminative embeddings. The visualization corroborates the findings from the confusion matrix; for instance, the clusters for phonetically similar words are located in close proximity within the embedding space. The overall structure of the plot is a strong indicator of a well-trained and powerful classifier.

3) *Error Analysis*: A qualitative inspection of misclassified samples, shown in Fig. 8, offers direct insight into the model's limitations. Some errors are phonetically plausible confusions, such as predicting "go" for "dog". More interestingly, other errors highlight the model's sensitivity to temporal artifacts;

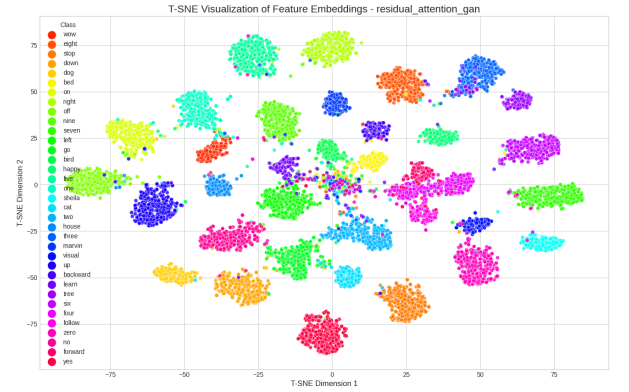


Fig. 7: T-SNE visualization of feature embeddings from the Experimental model.

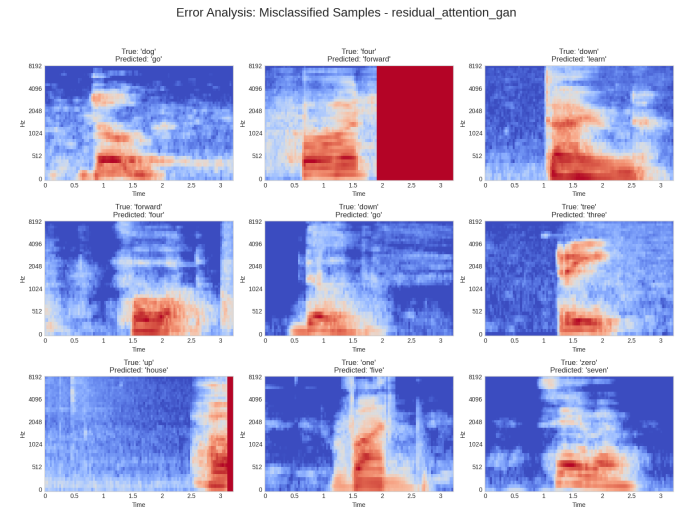


Fig. 8: Examples of misclassified samples by the the Experimental model.

for example, the prediction of "forward" for "four" shows a large silent region at the end of the spectrogram, corresponding to the silence padding added during preprocessing to standardize the length of audio clips, which may have confused

the classifier. This suggests that the model can be sensitive to utterances with unusual timing or significant trailing silence, highlighting a potential area for future improvement.

V. CONCLUSION

In this report, we presented a comprehensive comparative study of various deep learning paradigms for small-footprint keyword spotting. Through the implementation and evaluation of five advanced architectures and a baseline, we provided a detailed, multi-faceted analysis of the trade-offs between classification performance and model efficiency. Our work culminated in the proposal of a novel experimental architecture that achieved state-of-the-art accuracy in our tests, and the successful deployment of this model into a real-time Android application to validate its practical viability.

Our findings confirm that, while deep convolutional backbones are exceptionally effective for this task, a significant trade-off exists between raw accuracy and practical efficiency. Our experimental model achieved the highest F1-score of 0.947, demonstrating that a creative synthesis of architectural concepts can push performance boundaries. At the same time, models such as Hybrid CNN-BiLSTM proved to be remarkably efficient, delivering an almost equal F1-score of 0.945 with a dramatically smaller footprint. This landscape of results underscores a key lesson: The "best" model is highly context-dependent. For our deployment, we prioritized maximum classification accuracy and validated that our most complex model could be successfully implemented in a functioning application. However, in a real-world scenario, deploying the Hybrid CNN-BiLSTM model is preferable.

Several avenues remain for future work. First, leveraging pre-training on larger audio datasets could significantly boost performance, especially for data-hungry architectures like the Vision Transformer. Second, a promising direction would be to simplify our top-performing experimental model. By exploring techniques such as knowledge distillation or architectural pruning, it may be possible to retain most of its high accuracy while drastically reducing its size and latency, potentially creating a new model that dominates both in performance and efficiency. Finally, a deeper exploration of post-training optimization, such as full integer quantization, could yield even more compact models for highly resource-constrained hardware.

This project provided invaluable hands-on experience across the end-to-end machine learning lifecycle. The primary lesson learned is that academic metrics alone do not define a model's utility; success is a nuanced interplay between performance, efficiency, and the specific constraints of the application. The most challenging but rewarding aspect was to bridge the gap between a successful experiment and a useful tool by deploying the model on an Android device. Re-implementing the audio processing pipeline and ensuring fluid on-device performance taught us that much of the real engineering in machine learning lies in this final, critical step.

REFERENCES

- [1] T. N. Sainath and C. Parada, "Convolutional neural networks for small-footprint keyword spotting," in *Interspeech 2015*, pp. 1478–1482, 2015.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2017.
- [3] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," 2015.
- [5] R. Tang and J. Lin, "Deep residual learning for small-footprint keyword spotting," 2018.
- [6] J. Chorowski, D. Bahdanau, D. Serdyuk, K. Cho, and Y. Bengio, "Attention-based models for speech recognition," 2015.
- [7] D. C. de Andrade, S. Leo, M. L. D. S. Viana, and C. Bernkopf, "A neural attention model for speech command recognition," 2018.
- [8] L. Dong, S. Xu, and B. Xu, "Speech-transformer: A no-recurrence sequence-to-sequence model for speech recognition," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5884–5888, 2018.
- [9] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," 2021.
- [10] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "SpecAugment: A simple data augmentation method for automatic speech recognition," in *Interspeech 2019*, interspeech'2019, ISCA, Sept. 2019.
- [11] J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, "Squeeze-and-excitation networks," 2019.
- [12] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," 2016.
- [13] J. Zhu, X. Chen, K. He, Y. LeCun, and Z. Liu, "Transformers without normalization," 2025.
- [14] C.-H. Li, S.-L. Wu, C.-L. Liu, and H. yi Lee, "Spoken squad: A study of mitigating the impact of speech recognition errors on listening comprehension," 2018.
- [15] J. Six, O. Cornelis, and M. Leman, "TarsosDSP, a Real-Time Audio Processing Framework in Java," in *Proceedings of the 53rd AES Conference (AES 53rd)*, 2014.